The Operating System for Insurance

Craig Certo — April 2026

1. Vision

Origin

Every customer we build for, the same patterns emerge. The same architecture, rebuilt from scratch each time. EventGuard proved that AI can run an entire insurance program end-to-end — no humans in the loop. But every piece of it was custom. Standing up the next one means starting over.

The question became: what's the system underneath all of them?

Every Business Is a System

Every business, stripped down, is the same structure. Data with shape and lifecycle. People operating on that data in defined roles with defined permissions. When data changes, consequences flow — the right people are notified, the next steps are triggered, the external systems are updated. Every business runs this way. Most just don't see it as a single system because no single system models the whole thing.

Insurance companies run this structure on a patchwork of disconnected software. An AMS here, a carrier portal there, a rating engine, a CRM, email, fax. Each system holds a fragment. None of them model the whole business. And none of them were designed for a world where AI can participate in the work.

The operating system models the whole business. Every entity, every lifecycle, every role, every connection. And it makes every part of it operable by humans and AI through the same interface.

Six Building Blocks

The OS is six primitives. Not six features. Six structural building blocks that compose into anything.

Entities are the data — structured, with defined lifecycles and relationships. **Messages** are the connective tissue — when entities change, messages carry consequences to whoever needs to know.

Actors do the work, and the system draws no distinction between a human and an AI. **Roles** define what each actor can access and what changes flow to them. **Organizations** scope everything — each business is its own world on the same platform. **Integrations** connect the OS to every external system it touches.

A submission pipeline, a quoting engine, a policy administration system, a claims workflow, an embedded insurance product — each one is a composition of these six. Different entities, different roles, different rules. Same building blocks underneath.

Self-Evidence

This is what separates the OS from everything else.

When you define an entity — a Submission with fields, a lifecycle, relationships — you don't then build its API separately. You don't write its documentation. You don't create CLI commands for it. You don't teach an AI how to use it. You don't build a UI for it.

All of that exists the moment the entity is defined. The API. The CLI. The documentation. The AI interface. The human interface. The system describes itself, documents itself, and exposes itself to every surface — human, AI, programmatic — by the act of being defined.

Building a system on the OS is defining what the system IS. The rest follows.

AI as Architecture

AI today is an augmentation layer. Take an existing system, find a manual step, bolt on an AI tool. Draft some emails. Summarize some documents. Chat with customers. The AI sits on top, disconnected from the business logic, reaching in through whatever narrow interface was carved out for it.

On the OS, AI is architectural. An AI associate is an actor — the same kind of participant as a human employee. It has a role. It has permissions. It sees the same work queue. It uses the same operations. The system is indifferent to whether the actor is a person or an AI.

Put an AI alongside a human on the same role. Both see the same work. The AI handles what it can. The human handles what it can't. As the AI proves reliable, it takes on more. The human shifts to work that needs their judgment. The transition is a configuration change, not a rewrite.

This isn't a future where AI replaces people. It's a future where they work the same system, side by side, and the balance shifts naturally based on capability.

The Churning System

When the OS runs, it churns.

Something enters — an email, a phone call, a form, a webhook. It becomes an entity. Roles that watch for that kind of change are notified. Actors process the work — changing entities, creating new ones, transitioning states. Those changes notify more roles. More actors activate. The system moves continuously until every piece of work reaches a decision point or a final state.

There is no central workflow engine directing this. The behavior emerges from how roles are configured — what each role watches for, what each actor does when work arrives. The entire wiring of the system is visible at a glance and changeable through configuration. Add a step by adding a watch. Remove a step by removing one.

The OS isn't a set of static tools. It's a living system — humans and AI working together, messages flowing between them, the whole operation running like the organization it models.

What This Makes Possible

Setting up a new business on the OS is defining what that business is — entities, roles, rules, connections — not writing software. An AI agent can do this through the CLI. The system that runs businesses can be assembled by AI.

Any part of any workflow can have AI introduced through a role assignment. Not through a custom integration. A role assignment. Today a person does this job. Tomorrow an associate joins them. Next month the associate handles it alone. Role by role, across the entire operation.

Once a business runs on the OS, every piece of data is structured, every lifecycle is tracked, every operation is available through every surface. Intelligence compounds — every transaction improves the models, every pattern becomes a rule. The system gets smarter over time.

Products that couldn't exist become viable. A $200 umbrella policy that no agent will sell because the commission doesn't justify a phone call — an AI sells it in a conversation that costs fractions of a penny. Embedded insurance at checkout. Parametric coverage for niche markets. When the full lifecycle runs at near-zero marginal cost, the constraint isn't profitability. It's imagination.

And the kernel has no domain assumptions baked in. Insurance is where we start. The same six building blocks model any business.

The Future

Imagine every insurance company running on one platform. Operations modeled as entities and roles. Workflows emerging from configuration. Manual processes gradually automated by associates working

alongside employees.

Imagine acquiring a company and putting it on the OS in weeks. The cross-sell gap in their book — products every customer should have but nobody sells because the economics don't work for humans — mined immediately by AI. Legacy systems connected. Manual processes automated one by one. The company transformed not by a multi-year IT project but by modeling it on a platform that already exists.

Imagine a product engine that creates new insurance products through configuration. Filed with regulators, modeled on the OS, distributed through AI across every channel, iterated based on real data. Products that legacy carriers take eighteen months to launch, live in weeks.

This is what the operating system makes possible. The rest of this document is the blueprint.

---

2. Architecture

The Kernel

The OS has two layers: the kernel and the domain.

The kernel is the platform — the six primitives, the mechanisms that connect them, and the infrastructure that makes them work. It has no opinion about insurance. It doesn't know what a policy is or what an underwriter does. It knows about entities, messages, actors, roles, organizations, and integrations. It knows how to generate interfaces from definitions, evaluate conditions, route messages through watches, and execute work through actors. That's it.

The domain is everything built on top. The entities that model a specific business — submissions, policies, carriers, payments. The rules that encode that business's logic. The skills that instruct associates how to do the work. The watches that wire it all together for a specific operation. All of this is data, created through the CLI, stored in the database, changeable without touching the kernel.

This separation is what makes the OS domain-agnostic. The kernel is built once. Every business on the platform is a domain configuration on the same kernel.

Beyond the six structural primitives, the kernel provides seven **kernel entities** — entities the kernel itself depends on for core behavior:

Kernel Entity	Purpose
Organization	Multi-tenancy scope — the boundary around everything

Actor	Identity — humans, AI associates, and developers as participants
Role	Permissions and watches — what actors can do and what flows to them
Integration	External connectivity — providers, credentials, adapters
Attention	Active working context — who is attending to what, right now
Runtime	Execution environment — where associates actually run
Session	Authentication state — tokens, expiration, revocation

These are managed through the same CLI and API as any domain entity. They just happen to be entities the kernel needs to function.

Entity

An entity is the fundamental unit of the OS. It's data with structure — typed fields, a state machine that enforces valid transitions, relationships to other entities, and operations that can be performed on it.

When an entity type is defined, the kernel generates its full surface automatically. The CLI commands to create, read, update, delete, transition, and invoke operations. The API endpoints that mirror those commands. The documentation — a skill in markdown — that describes every field, every state, every operation, every relationship. The UI representation. All derived from the definition.

This is the self-evidence property. The entity definition is the single source of truth. Change the definition and every surface updates — the CLI gains new commands, the API gains new endpoints, the documentation reflects the new fields, the UI renders the new shape. No separate maintenance of interfaces or docs.

Entities have **state machines** — optional, activated when defined. The kernel enforces that only valid transitions can occur — an entity can't skip states or move backward through its lifecycle unless the definition explicitly allows it. This is structural integrity, not business logic — it applies universally across every organization.

Entities have **computed fields** — values derived from other fields. For example, an entity might compute a responsibility assignment from its current stage, or a staleness flag from its last activity timestamp. These are declared in the entity definition and maintained automatically by the kernel.

Entities have **flexible data** — a section for fields that vary by product or configuration. The core structure of an entity type is the same everywhere. Product-specific or customer-specific fields live in a flexible section governed by a schema. For example, two different insurance products might require

different application fields, but the entity structure that manages them is identical. Fixed structure where it matters. Flexibility where it's needed.

Schema migration is first-class. Entities evolve — fields are added, renamed, restructured. The kernel handles this with batching, dry-run previews, aliases during migration windows, progress tracking, audit trails, and rollback. Additive changes like new fields need no migration at all.

Message

A message is the consequence of an entity change.

When an entity is created, when it transitions states, when an exposed operation is invoked on it — the kernel evaluates the watches configured across all roles in that organization. For each watch that matches, a message is written to the queue targeting the matching role.

This happens atomically. The entity save, the watch evaluation, and the message writes are a single transaction. If any part fails, none of it commits. There is no window where an entity changed but the message wasn't created, or where a message exists but the entity didn't actually change.

Selective emission keeps the message volume sane. Not every field change generates messages. Only state transitions, exposed operations, and entity creation and deletion. An associate updating ten fields during processing doesn't generate ten messages. The save at the end of the operation generates one, capturing everything that changed.

One save, one event. A single entity save produces one event with metadata about what changed — which operation was invoked, which fields changed, which state was transitioned to. Watches evaluate against this metadata. Each matching watch produces one message. The event is the unit. Not the individual field change.

Messages are stored in two collections — a hot queue for active work and a cold log for completed items. The queue stays small and fast regardless of history. The log retains everything for audit and analysis.

Messages carry **entity references**, not copies. The entity in the database is the source of truth. When an actor processes a message, it loads the current entity state — fresh, authoritative. No stale data.

Every message carries a **correlation ID** — a trace identifier that links it to the original event that started the cascade. For example, a single inbound email might trigger a chain of six actors across classification, linking, assessment, drafting, and review — every message in that chain shares the same correlation ID. The entire cascade is reconstructable.

Watches

Watches are the wiring mechanism. They live on roles and declare what entity changes matter to actors in that role.

A watch is: an entity type, an event type, and optionally a condition. For example, a watch might declare "when this entity type is created where a specific field equals a specific value." Any actor with the role receives a message when that condition is met.

Watches use a **single condition language** — structured JSON with field comparisons and logical composition (all, any, not). The same language is used for watch conditions and for rules. One evaluator, one syntax, one debugging surface.

Watch conditions are **entity-local** — they can only reference fields on the entity that changed. No cross-entity lookups during the save transaction. If a watch needs to consider a related entity's state, either the relevant field is denormalized onto the entity, or the actor evaluates the condition after claiming the message. This constraint keeps watch evaluation fast and predictable.

Watches support **scoping** — the ability to target a specific actor rather than everyone in the role. Two resolution types:

Field path scoping resolves an actor reference by traversing entity relationships. For example, a watch can be scoped so that only the actor who owns a related entity receives the message — used for ownership patterns where one actor cares about events on their entities, not everyone's.

Active context scoping resolves by looking up which actors are currently attending to a related entity. For example, when a payment completes during a live conversation, active context scoping delivers that event to the specific associate holding the conversation — not to every associate in the role. Used for real-time event delivery without delay.

Scope is resolved at emit time. The kernel writes `target_actor_id` on the message when it's created. Claim queries filter by it. No runtime scope evaluation.

The entire behavior of a system on the OS is the set of watches across all roles. It's visible, configurable, and self-describing. No hidden orchestration logic.

Actor

An actor is a participant in the system. The kernel distinguishes actors by identity and role, not by whether they're human or AI.

Every actor has one or more roles. The roles determine what the actor can access (permissions) and what work flows to them (watches). The actor's queue is the union of messages matching all their roles' watches.

The actor spectrum. Associates — AI actors — exist on a spectrum from fully deterministic to fully reasoning. A deterministic associate follows a fixed procedure: run this command, check this condition, run the next command. A reasoning associate reads its skill, analyzes the situation, and decides what to do. A hybrid does both — tries the deterministic path first, falls back to reasoning when the deterministic path can't handle it.

The OS doesn't care where on this spectrum an actor falls. It sees the same thing regardless: a trigger arrives, CLI commands are executed, entities change, messages are generated. What happens between the trigger and the commands — whether a human thought about it, an LLM reasoned about it, or a script ran through it — is the actor's business.

The unified queue. The message queue is universal. Every message targeting a role is visible to every actor in that role — human or AI. Associates claim from the queue through durable workflows. Humans claim through the UI. Same messages, same claiming mechanism.

This is what makes gradual rollout possible. Add an associate to a role alongside a human. Both see the same queue. The associate claims what it can process. The human handles the rest. If the associate fails, the message returns to the queue. If the associate gets it wrong, the human corrects it. Remove the human from the role when the associate is trusted. The infrastructure never changes — only the role assignments.

Role creation has two paths. Named shared roles — like "underwriter" or "operations" — are organizational job functions that humans hold and associates may share. They're reusable, grantable, and visible as named concepts. Inline roles are created alongside an associate for automation-specific scope — permissions and watches that only that associate needs. Same primitive, two ergonomics.

Capabilities and the —auto Pattern

The OS ships with a library of **kernel capabilities** — reusable operations that any entity can activate. Auto-classify. Fuzzy search. Pattern extraction. Stale detection. Auto-link. Auto-route. Rule evaluation.

These are code in the kernel, activated on entity types through the CLI, and parameterized by per-organization configuration — rules, lookups, thresholds. The capability is universal. The configuration makes it specific to each business.

Rules are per-organization condition-action patterns. When an entity passes through a capability's evaluation, rules are checked against its fields. For example, a rule might match on a field value and set a classification — entirely deterministic, no AI involved. Rules use the same condition language as watches — structured JSON with composition.

Rules have exactly two actions: **set fields** and **force reasoning**. Set-fields applies a deterministic result. Force-reasoning overrides a positive match and sends the case to the AI — used for veto rules that catch cases where the deterministic path would get it wrong.

Rules are organized into **groups** with a lifecycle — draft, active, archived. New rules can be tested before they affect production. Conflict detection at creation time prevents overlapping rules without explicit resolution. This organizational structure is necessary from day one to prevent rules from becoming unmanageable as they grow.

Lookups are mapping tables — separate from rules to prevent rule explosion. For example, instead of writing dozens of rules to map individual codes to categories, a single lookup table handles all the mappings. Lookups can be bulk-imported from CSV and maintained by non-technical users. Rules reference lookups when they need to resolve a value.

The **`--auto` pattern** ties it together. When an associate invokes an entity method with `--auto`, the kernel tries the deterministic path first — evaluates rules, checks lookups, applies the result if a match is found. If no match, or if a veto rule fires, the method returns `needs_reasoning` with context about why the deterministic path failed. The associate's skill provides the AI fallback.

This means the cost of AI processing is proportional to the complexity of edge cases, not the volume of routine work. And the system gets more deterministic over time — every pattern the AI keeps handling is a candidate for a new rule. The `needs_reasoning` rate is the metric that tells you where to add rules.

Skills

Skills are markdown documents. Always.

There are two kinds. **Entity skills** are auto-generated from entity definitions — they document what CLI commands exist, what fields an entity has, what states it can be in, what operations are available. They're reference material. **Associate skills** are written by humans or AI — they describe how to orchestrate entity operations for a specific kind of work. When to classify. When to link. When to escalate. When to use `--auto` and what to do when it comes back `needs_reasoning`.

Associates load their skills at startup. The skills are the program. The associate's execution mode — deterministic interpreter or LLM — is what runs the program. The OS doesn't care which interpreter is used. It sees the same outcome: CLI commands executed, entities changed.

Skills are stored in the database, versioned through the changes collection, and updatable through the CLI. Updating a skill takes effect immediately. The changes collection records every version. Rollback is a CLI command.

Integration

An integration is the kernel's representation of an external system connection. It holds the provider, the configuration, the credential reference, and the lifecycle state.

Adapters are the implementation. Each provider has an adapter — kernel code that knows how to talk to that specific external system. For example, an email integration points to a specific provider, and that provider's adapter knows how to fetch messages, send messages, and validate inbound webhooks using that provider's API. New providers mean new adapters. Everything else — the integration entity, the CLI, the credential management — is the same.

Integrations support both **outbound and inbound** connectivity. Outbound: fetch emails, charge a payment, submit to a carrier. Inbound: receive a Stripe webhook, accept an OAuth callback, handle an identity provider redirect. The kernel provides a generic webhook endpoint that dispatches to the integration's adapter for validation and parsing.

Credential resolution follows a priority chain. When an entity operation needs an external system, the kernel checks: does the calling actor have a personal integration of this type? If so, use it. If not, does the organization have a shared integration that the actor's role permits? If so, use it. Otherwise, fail with a clear message.

This dual ownership model — org-level for shared connections, actor-level for personal ones — handles both cases through the same primitive. For example, a company's shared carrier API and an individual employee's personal email are both integrations, just with different owners.

Credentials never live in the database. Integration entities store a reference to an external secrets manager. Rotation, audit, and access logging are first-class operations on the integration.

Associates acting on behalf of humans. An associate can be bound to a human actor through an owner reference. When the associate needs an external connection, the credential resolver checks the owner's personal integrations in addition to the associate's own. For example, a scheduled sync associate bound to a specific team member uses that team member's personal email integration — with their consent, auditable, and revocable. This enables personal sync associates, the default UI assistant, and any pattern where an associate acts on behalf of a human using their credentials.

Content visibility for personal integrations. When an entity is created from a personal integration's data, a privacy question arises: who can see the full content? The integration carries a visibility policy — metadata shared with the team (who contacted whom, when, about what), full content scoped to the owner (the actual email body stays private). This is configurable per integration. The default for personal integrations is metadata shared, full content owner-scoped.

Attention, Runtime, and Real-Time

Three kernel entities support real-time operation.

Attention represents an actor's active working context. "This actor is currently attending to this entity, with this purpose, until this time." It unifies presence awareness (who's looking at this submission right now), soft-locking (warn others that someone is actively reviewing), and real-time event routing (deliver mid-conversation events to the running associate).

Attention records are heartbeat-maintained with TTL expiration. If heartbeats stop — the browser tab closes, the associate crashes — the attention expires automatically. Five purposes: real-time session, observing, review, editing, claim in progress.

Runtime is a deployable host for associate execution. It describes the execution environment — the framework, the transport, the capacity, the deployment image. One runtime hosts many associates. The associate carries the per-session configuration (skills, model, mode). The runtime provides the environment.

The **harness pattern** bridges a specific agent framework and transport to the OS. Each harness is a thin piece of glue code — roughly 60 lines — that loads the associate's configuration at session start and uses the CLI for all OS interactions. For example, a voice harness bridges a voice framework to the kernel, and a chat harness bridges a WebSocket server. The harness has full access to its framework's capabilities. The kernel doesn't constrain it.

Handoff between actors on a live interaction is a field update. The interaction entity has a handling actor. Transfer changes the handler. The runtime notices and switches modes — from running the associate in-process to bridging turns through the queue for a human, or vice versa. Observation is a first-class state — a human can watch a live interaction before deciding to take over.

When a human takes over a voice interaction, their voice client is an Integration — the same primitive used for email and payment connections, just with a voice client provider type. No new concept needed for human participation in real-time channels.

Three-layer customer-facing flexibility. The behavior of a customer-facing surface is configured across three layers: the **Deployment** entity controls transport behavior — branding, widget appearance, greeting. The **Associate** skill controls conversation style — what the agent says, how it handles turns. The **Runtime** controls the execution environment — model, framework, capacity. Per-session overrides merge across layers. This flexibility exists in the entities already — it's a pattern, not a mechanism.

Everything Is Data

The OS codebase is the platform. The database is the application.

Entity definitions, skills, rules, lookups, role configurations, associate configurations, capability activations — all stored in the database, all managed through the CLI. The kernel code provides the framework. The database provides the content.

This means:

Environments are organizations. Development, staging, and production are different orgs on the same kernel. Clone a production org to create staging. Diff the two to see what changed. Deploy from one to the other. Roll back if something breaks.

Version control is built in. The changes collection records every mutation to every configuration object — entity definitions, skills, rules, roles, associates, integrations. Every change is timestamped, attributed to an actor, and includes before and after values. History is queryable. Diffs are available. Rollback is a CLI command.

AI can build systems. Because everything is data managed through the CLI, an AI agent can set up an entire business: define entities, configure rules, create roles with watches, deploy associates with skills, connect integrations. The system that runs businesses can be assembled autonomously.

Seed templates bootstrap new organizations. The OS ships with a standard library of entity definitions, default skills, and reference roles as seed files. When a new organization is created, it can clone from a template — getting a working starting point that can then be customized. Updates to the standard library can be pulled selectively by existing organizations.

Security

Organization isolation is enforced at the database access layer. All queries pass through a scoped wrapper that always injects the organization boundary. Raw database access is hidden inside kernel initialization. Application code cannot construct an unscoped query. Cross-organization access for platform administrators uses a separate accessor with full audit.

Credentials never live in the database. Integration entities store references to an external secrets manager. The kernel's adapter resolution code reads credentials at runtime. They never appear in entity queries, CLI output, or API responses.

Skills are tamper-evident. Content hashes are computed on creation and verified on load. A skill modified outside the normal update path is rejected. Skill updates go through a version approval workflow — new versions can be reviewed before activation.

The audit trail is append-only. The changes collection records every entity mutation with field-level detail — who changed what, when, from what value, to what value, and why. A sequential hash chain links records together. Tampering breaks the chain. Verification is a CLI command.

Rule validation prevents injection. State machine fields cannot be set directly by rules — transitions must go through the state machine. Fields in rule actions are validated against the entity schema. The creating actor must have write permission for every field the rule touches.

Observability

Observability is built into the kernel, not bolted on afterward.

Every entity save, every watch evaluation, every rule evaluation, every associate invocation, every CLI command — instrumented with traces and spans. The correlation ID on messages is the trace ID. A single cascade — for example, an inbound event that triggers five actors in sequence — is one trace with nested spans for every step.

Three data stores serve three purposes, linked by the same trace ID:

Store	Records	Retention	Query Pattern
Changes collection	Field-level entity mutations — who, what, when, from, to	Years (regulatory)	By entity
Message log	Completed work items — who processed what, when, result	Months to years (operations)	By role, date, entity type
Trace backend	Full execution paths — timing, errors, parameters	Days to weeks (debugging)	By trace ID, span attributes

Not redundant. Three optimized views of the same events, each serving a different audience — compliance, operations, engineering — linked by one identifier.

---

3. Domain Modeling

The Process

The architecture defines the kernel — the six primitives and the mechanisms that connect them. Domain modeling is how you use the kernel to represent a specific business. It's the bridge between "the OS exists" and "this company runs on it."

The process was formalized through the design sessions and validated against three very different workloads. It applies to any business being modeled on the OS.

Step 1: Understand the business. What does this company actually do day to day? What are the workflows? Who are the people? What systems do they use? What's painful? The output is a narrative understanding — not a technical document. You can't model what you don't understand.

Step 2: Identify the entities. What are the nouns? What data does this business create, manage, and act on? For each entity: what are its fields, what is its lifecycle, what are its relationships to other entities? The test: can you describe the business entirely in terms of these entities and their states?

Step 3: Identify the roles and actors. Who participates? What types of participants exist? For each role: what entities can they access, and what entity changes do they care about? Which roles are humans? Which could be associates? Which are both? The test: for every entity state change in the business, there should be a role whose watch catches it.

Step 4: Define the rules and configuration. What per-organization business logic exists? Classification rules, routing thresholds, required fields, lookup tables. These parameterize the kernel capabilities activated on the entities. The test: can you trace through the common case entirely with rules, no AI reasoning needed?

Step 5: Write the skills. For each associate role: behavioral instructions in markdown. What CLI commands does it use? When does it follow a procedure and when does it use judgment? The test: can a human reading the skill understand what the associate does?

Step 6: Set up integrations. What external systems does this business use? For each: is it org-level or actor-level? Which adapter exists in the kernel? If no adapter exists, that's a kernel contribution — new adapter code that then becomes available to every business on the platform.

Step 7: Test in staging. Create a staging organization, apply all configuration, run the pipeline with realistic data. Validate that watches fire correctly, associates process as expected, and humans see the right work in their queues.

Step 8: Deploy and tune. Promote to production. Monitor. Tune rules based on real data — the `needs_reasoning` rate tells you which deterministic rules are missing. The system gets more deterministic over time as rules are added for patterns the AI keeps handling.

The Universal Pattern

Every system modeled on the OS follows the same structural pattern:

```
Entry point (email, webhook, chat, form, API call, schedule)
  → Creates or updates an entity
    → Watches fire
      → Actors process (deterministic first, reasoning if needed)
        → Entity state changes
          → More watches fire
            → Eventually reaches a human decision point or a final state
```

Domain modeling fills in the specifics: WHICH entities, WHICH watches, WHICH rules, and WHICH skills populate this universal structure for a given business. The pattern itself never changes.

What Varies vs. What's Universal

Universal (kernel)	Per-business (domain)
Entity framework — structure, lifecycle, CLI/API generation	Which entities, which fields, which states
Watches as the wiring mechanism	Which watches on which roles
Rules and lookups evaluator	Which rules, which thresholds, which mappings
Kernel capabilities library	Which capabilities activated, with what configuration
Actor model — queue, claiming, execution	Which roles, which associates, which skills
Authentication, sessions, permissions	Which auth methods, which MFA policy
Integration primitive and adapter dispatch	Which providers, which credentials

The kernel is built once. The domain modeling process is repeated per business. And it compounds — reusable components grow over time. Standard entity templates emerge from repeated use. Proven skills get shared. Common rule patterns become starting points. Existing adapters cover more and more providers. Each new business modeled on the OS is faster than the last.

Domain-Agnostic by Design

The kernel has no domain assumptions. It doesn't know what insurance is. It doesn't know what a policy is or what an underwriter does. It knows entities, messages, actors, roles, organizations, and integrations.

This was validated by design — during the architecture process, we traced three radically different workloads through the kernel. A B2B insurance email processing pipeline. A consumer-facing real-time insurance transaction with no humans in the loop. And a generic B2B customer intelligence system with zero insurance concepts. Same six primitives. Zero kernel changes between the three. The third workload proved that the kernel works with no insurance-specific entities, roles, or rules whatsoever.

Insurance is the first domain. It is not the only domain the kernel supports. Any business that can be described as entities with lifecycles, actors with roles, and messages flowing between them can run on this OS. The domain modeling process is the same regardless of industry.

Internal Actors vs. External Entities

An important distinction in domain modeling: **actors** and **external counterparties** are different things.

Actors authenticate with the OS and have roles. They are participants in the system — the humans, associates, and developers who do work. They appear in the actor list, hold roles, see queues, and are audited.

External counterparties — for example, the customers buying insurance, the retail agents submitting applications, the partner carriers providing products — are **entities**, not actors. They have data and lifecycle like any entity, but they don't authenticate, don't have roles, and don't participate in the message queue. They're the subjects of the business, not participants in the system.

Both concepts are valid and coexist. The kernel handles actors through the actor/role primitives. The domain handles external counterparties through entity definitions. Confusing the two — for example, trying to make every customer an actor — would break the model.

Entry Points

External events enter the OS by creating or updating entities, after which the kernel takes over.

Entry Point	What Happens
Channel (web chat, voice, SMS)	Interaction entity created, channel stays open for ongoing communication
Webhook (inbound HTTP)	Adapter validates and parses the payload into entity operations
API call	Entity created or updated directly
Polling (scheduled fetch)	Integration adapter pulls data, creates entities from what it finds
CLI command	Entity created or updated directly

Form submission	Entity created from submitted data

Once an entity exists or changes, watches evaluate, messages flow, and actors process. The entry point determines how data gets in. The kernel determines what happens next.

Actor Triggers

Actors activate through three trigger types:

Trigger	When It Fires
Message	A watch on the actor's role matches an entity change
Schedule	A cron expression fires, creating a message in the queue — same path as watch-triggered work
Direct invocation	Explicit CLI or API call — used for real-time channels where latency matters

Every actor activation is one of these three. Channels like voice and chat feel like they should be separate trigger types, but they're actually entry points that create entities. The associate's trigger is still a message — its watch matches the new entity. For real-time channels, direct invocation bypasses the queue for latency while still creating a queue entry for visibility and audit.

---

4. Authentication

Design Principles

Authentication follows the same principles as the rest of the kernel: compose from existing primitives, store everything as data, keep credentials out of the database, and audit everything.

The design was driven by ten forcing functions — specific scenarios that the authentication system must handle cleanly. These range from a new team member joining, to an enterprise customer enabling SSO, to a Tier 3 developer signing up self-service, to a stolen laptop requiring immediate session revocation. If the design handles these ten scenarios, it's sufficient.

Session

Session is the seventh kernel entity. It represents an authenticated presence in the system — a logged-in human, a running associate, a Tier 3 developer's API connection, or a CLI automation script.

Every authenticated identity has a Session. One validation path, one revocation mechanism, one audit trail, regardless of who or what is authenticated. Sessions carry a type that distinguishes how they were created:

- **User interactive** — a human logged in through the UI or CLI, with short-lived access tokens and long-lived refresh tokens
- **Associate service** — an AI associate authenticating as itself with a long-lived token
- **Tier 3 API** — a developer's machine-to-machine connection
- **CLI automation** — headless scripts using long-lived tokens

The access model is hybrid. Short-lived access tokens are verified without hitting the database — stateless, fast, on every request. Long-lived refresh tokens are stored securely and validated on refresh. For associates and API connections, long-lived tokens are validated by lookup. One model covers all cases.

When a session is revoked — for example, because a laptop was stolen — every API instance picks up the revocation within seconds through a change stream. All of that user's tokens fail on the next request. No delay, no window of vulnerability.

Authentication Methods

Every actor carries a list of authentication methods. Multiple methods can coexist on the same actor — for example, password plus MFA, or SSO with password as a fallback for emergencies.

Five method types:

Method	How It Works
Password	Kernel-native. Hash stored in the database (non-reversible). The password itself is never stored.
TOTP	Kernel-native. Time-based one-time password for MFA. Shared secret stored in the external secrets manager.
SSO	Via integration. The identity provider is an Integration entity — same primitive used for email providers and payment processors. Login redirects to the provider, validates the returned token, and issues an OS session.
Token	Kernel-native long-lived opaque token. Used for associates, Tier 3 API keys, and CLI automation. Hashed and stored. Validated by lookup.

Magic link	Kernel-native one-time token for invitations and password recovery. Sent via email, deactivated after use.

No credentials are stored inline in the database. Everything references the external secrets manager. The kernel reads credentials at validation time and never exposes them through queries, CLI output, or API responses.

SSO and password can coexist in the same organization. For example, an enterprise customer might use SSO as the standard login flow while retaining password access for administrators as an emergency fallback if the identity provider goes down.

MFA Policy

MFA requirements are set at the role level, with actor-level overrides and an organization-level default.

A role can declare that MFA is required — meaning any actor holding that role must complete MFA during login. An individual actor can be marked MFA-exempt for accessibility or emergency reasons, which is audited. An organization can set a default MFA requirement that applies when roles don't specify.

The resolution order: actor exempt overrides role required overrides organization default. This matches operational language — "all underwriters must have MFA" is a role-level setting. "This specific actor is exempt" is an actor-level override.

For sensitive operations — for example, rotating credentials or granting administrative roles — the architecture supports fresh MFA re-verification, where certain operations can require that MFA was verified within the last N minutes. This is deferred beyond MVP, where session-level MFA verification is sufficient.

Platform Admin Cross-Organization Access

The OS is designed for a model where Indemn builds most things for the customer. Platform administrators — forward-deployed engineers and implementation staff — routinely work inside customer organizations to configure entities, rules, skills, and integrations.

Platform admin sessions are:

- **Time-limited.** Default four hours, maximum twenty-four. Auto-renewable on activity with an idle timeout.
- **Work-type tagged.** Each session is categorized — build, debug, incident response, or routine. This informs audit reports and customer notifications.

- **Customer-visible.** Each customer organization configures how it wants to be notified of platform admin activity — on session start, on every operation, as a daily summary, or audit-only.
- **Scope-limited.** Platform administrators cannot read secrets, cannot modify credentials except through audited rotation flows, and cannot impersonate customer users. The session identity stays as the Indemn actor. Every operation is attributed to them.
- **Audited in the customer's org.** Every action is recorded in the target organization's changes collection with full provenance — who, from where, what type of work, why.

Role Changes and Session Handling

When an actor's roles change mid-session, the system handles it without disrupting the user.

If a role is granted, the new permissions take effect on the next natural token refresh — within minutes. If a role is revoked, the system marks active sessions as needing a claims refresh. The next request triggers an automatic refresh with the updated role set. The user sees their permissions change on the next interaction without being kicked to a login screen.

Full session revocation — forcing a complete re-login — only happens on actor suspension or deprovisioning.

Recovery Flows

Three recovery scenarios, each with a distinct mechanism. No back doors. No security questions. No SMS-based recovery.

Password reset. User requests a reset, receives a magic link via email. Clicking it verifies the token, prompts for a new password. All existing sessions are revoked as a security measure.

MFA recovery. Two paths. First: backup codes generated at MFA enrollment — pre-generated, single-use, stored hashed. If the MFA device is lost, a backup code completes authentication and prompts for re-enrollment. Second: if backup codes are also lost, an administrator with grant authority on the actor's role can reset MFA, forcing re-enrollment on next login.

Emergency access. If an organization's last administrator is fully locked out, platform admin intervention is required — a short-duration incident session with full audit and customer notification.

Tier 3 Self-Service Signup

Developers can sign up through the website. The flow creates an organization, a first admin actor, a password method, and a verification email. After verification, the developer receives an API key and

access to the developer portal. Billing, plan selection, and team invitation are deferred beyond MVP.

First-Organization Bootstrap

The very first organization on the platform has no email integration yet — there's nothing to send invitation emails through. The bootstrap flow prints a one-time token to the console. The first administrator uses it to complete setup. After configuring the platform's email delivery integration, all subsequent invitations use email. Same mechanism — magic link — different delivery channel for the bootstrap case.

Authentication Audit

Every authentication event flows through the same change-tracking system as entity mutations. Login attempts, session creation, session revocation, MFA challenges, password changes, role grants, platform admin access, brute-force lockouts — all recorded with specific event types in the changes collection. Same tamper-evident hash chain. Same queryable history. Surfaced through the base UI and CLI.

---

5. Base UI

Design Principle

The base UI is not a dashboard built on top of the OS. It is the OS rendered visually.

When you define an entity, it appears in the UI. When you activate a capability on an entity, the corresponding action appears as a button. When you create a role with watches, actors in that role see their queue populated. When you add a field to an entity, the forms and tables reflect it. The UI derives from the system definition. You don't build the UI. You build the system and the UI follows.

This is the same self-evidence property that generates the CLI and API from entity definitions — extended to the visual surface. One source of truth. Every surface reflects it.

What It Renders

The base UI auto-generates from primitives. Here is what produces what:

Entity list views. Interactive tables for each entity type the current role can read. Columns derived from field types. Filters from common fields. Row actions from state transitions and exposed operations the

role can invoke. Sorting, pagination, and filtering built in.

Entity detail views. Forms showing all fields with appropriate controls — text inputs, date pickers, dropdowns for enums, entity pickers for relationships. The current state displayed as a badge. Available transitions shown as buttons. Exposed operations shown as actions. Related entities linked for drill-through. Recent changes for this entity shown from the audit trail.

Queue view. The actor's home screen — pending messages targeting their roles, sorted by priority and age. Each row shows a summary of the entity and the event that triggered it. Row actions let the actor claim, complete, or defer work. Coalesced batches — for example, when a scheduled operation updates many entities at once — render as a single row with drill-down to individual items.

Role overview. Aggregate counts for each role — queue depth, active attentions held by actors in the role, completion rate, pointers to bottlenecks. A quick health check for how work is flowing through the system.

Kernel entity views. The seven kernel entities — organizations, actors, roles, integrations, attentions, runtimes, sessions — are rendered with the same auto-generation as domain entities. Operations staff see them as normal tables with row actions like pause, drain, rotate credentials, close attention, or transfer interaction.

Pipeline metrics. For entities with state machines, the UI renders state distribution — how many entities are in each state — plus throughput over time windows and dwell time per state. These are kernel-provided aggregation capabilities, not custom dashboards. The data comes from the same entity queries and message log that everything else uses.

Tables Over Charts

The default visualization is interactive tables. Charts are reserved for continuous time-series data where the shape of a trajectory matters — and even then, a table with counts and deltas is usually clearer. A count per state is a table, not a bar chart. Throughput per hour is a table comparing windows, not a line chart.

Tables are interactive — sort, filter, drill into rows, take actions. Charts aren't. The primary data surface should be the one you can act on.

Forms Are Auto-Generated

Entity creation and update forms derive from the entity definition. Field types map to form controls. Required fields are enforced. State machines are shown as the current state plus available transition buttons — not as a diagram. Simpler is better.

The Assistant

Every human actor has a default associate — an AI assistant pre-provisioned with the same permissions as the user. The base UI surfaces this assistant as a primary interface element: a persistent, always-visible input at the top of every view. Not a sidebar. Not a modal. Not a hidden keyboard shortcut. A first-class input surface.

The assistant operates through the same CLI and API as every other associate. When a user asks it something, it has implicit context from the current view — what entity they're looking at, what filters are applied, what role they're in. It can execute any operation the user has permission for. It can answer questions about the data, take actions, and report back with inline references to the entities involved.

The assistant inherits the user's session — it authenticates as the user, and every action it takes is audited as "user via assistant." When the user logs out, the assistant's session ends. This is distinct from scheduled associates that run independently with their own tokens.

The conversation is persistent per user. The assistant remembers context across turns within a session. It subscribes to real-time events relevant to the current view, so if an entity changes while the user is looking at it, the assistant knows immediately.

Real-Time By Default

Views subscribe to entity changes in real time, filtered by the current query and pagination. List views, detail views, queue views, and kernel entity views all update live. Users see the system churning — entities transitioning states, queue items arriving and being claimed, integrations reporting health. The UI is a live window into the running system, not a series of periodic snapshots.

What the Base UI Is Not

It is not a bespoke dashboard application with per-organization custom views. No custom UI code per entity. No custom UI code per organization. The auto-generation covers the operational surface. Configurable widgets and custom dashboards may be added in the future when forcing functions demand it, but for the initial system, auto-generation from primitives is the approach.

It is not the customer-facing UI. The base UI is for internal operators — Indemn staff, forward-deployed engineers, customer operations teams. Customer-facing interfaces — policyholder portals, agent workbenches, embedded widgets — are separate products built on the same primitives and API. The base UI is the operations surface.

Active alerting is deferred. The base UI provides visibility — operators watch the system through the live views. Active alerts — notifications when thresholds are breached, integrations fail, or queues grow — will be added when specific thresholds and notification patterns are established through real usage. The mechanism will likely be watches on kernel entities with actions that invoke notification integrations.

---

6. Bulk Operations

The Pattern

Bulk operations are a kernel-provided pattern, not a separate primitive. The six structural primitives and seven kernel entities are unchanged. Bulk operations compose from what already exists — messages for queue visibility, durable workflows for execution, transactions for per-batch atomicity, selective emission for event discipline, and the changes collection for audit.

The need is concrete. Scheduled processes that update dozens or hundreds of entities at once. One-time imports from CSV. Data migrations across entity types. Reprocessing after rule changes. Each of these needs: batch execution with progress tracking, per-entity event emission, failure handling, idempotency, and visibility into what happened.

Execution

A generic durable workflow handles all bulk operations. It receives a specification — entity type, operation, source data, batch size — and processes entities in batches. Each batch is a single transaction. Progress is tracked through the workflow state. The operation is resumable — if the worker crashes mid-batch, the workflow replays from the last completed batch.

Three entry points, one mechanism:

- **CLI commands** auto-generated per entity type. For example, `bulk-create`, `bulk-transition`, `bulk-method`, `bulk-update`, `bulk-delete`. These are thin wrappers that construct the specification and start the workflow.
- **Scheduled associates** whose skill invokes a bulk CLI command. The kernel makes no distinction — the schedule fires, the skill runs a bulk command, the workflow executes.
- **Direct invocation** by any actor through the CLI.

Event Emission

Bulk operations preserve the one-save-one-event rule. Each entity write produces its own event. A bulk operation updating 50 entities produces 50 events, each carrying context linking it back to the bulk operation.

The UI handles the volume through grouping — events from the same bulk operation share a correlation ID and render as a single row with drill-down. Individual events are still claimable by actors who need to process them individually.

CLI Verb Taxonomy

The CLI verbs enforce the selective emission discipline:

Verb	Purpose	Emits Events?
`bulk-create`	Create entities from source data	Yes — creation events
`bulk-transition`	Transition entities through state machines	Yes — state change events
`bulk-method`	Invoke an exposed operation on each entity	Yes — method events
`bulk-update`	Raw field updates — migrations and backfills	**No** — silent
`bulk-delete`	Delete entities (gated, dry-run required)	Yes — deletion events

The distinction matters. If a change should cascade through watches and trigger downstream actors, it goes through `bulk-transition` or `bulk-method`. If it's a data migration that shouldn't trigger anything, it goes through `bulk-update`. The verb choice makes the intent explicit.

Failure Handling

The default failure mode is skip-and-continue. Permanent errors — validation failures, permission denials, entities not found — are recorded and skipped. Transient errors — network issues, lock timeouts — trigger a retry of the batch. The workflow reaches a terminal state of `completed`, `completed_with_errors`, or `failed`.

For operations requiring strict all-or-nothing semantics, an abort mode is available — the workflow stops at the first error. Dry-run is the safety net for destructive operations: preview what would happen before committing.

Idempotency is at the entity level. State machines are naturally idempotent — transitioning an already-transitioned entity is a no-op. For creation, external references prevent duplicates. For method

invocations, the skill author ensures idempotent behavior. This makes the workflow safe to replay on partial failure.

---

7. Infrastructure and Deployment

Service Architecture

The OS runs as five services from one kernel image plus additional service types.

Three kernel processes share one image with different entry points:

- **API Server.** The gateway. REST API, WebSocket for real-time UI updates, webhook endpoints, authentication endpoints. The only service that faces the internet. Every CLI command, every UI interaction, every harness operation, every Tier 3 API call goes through it.
- **Queue Processor.** A lightweight internal sweep. Catches undispatched workflows, runs escalation checks, creates scheduled queue items. Not the primary dispatch path — a reliability backstop.
- **Temporal Worker.** Executes associate workflows — the generic claim-process-complete cycle, bulk operations, platform deployments. Connects to the durable workflow engine for crash recovery and retry.

Two additional service types:

- **Base UI.** A static application served independently. Connects to the API server only — no direct database access.
- **Harnesses.** One image per runtime kind and framework combination. For example, a voice harness or a chat harness. Each connects to the API server through internal networking and uses the CLI for all OS operations.

The Trust Boundary

A clean split between what has direct database access and what doesn't.

Inside the trust boundary: the API Server, Queue Processor, and Temporal Worker. These three are the kernel. They share the same codebase, the same image, and the same database credentials.

Outside the trust boundary: the Base UI, harnesses, the CLI, and Tier 3 applications. All of these access the system through the API, authenticated with tokens. Permissions are enforced by the API's auth middleware on every request.

This means only three processes need database credentials. Everything else authenticates to the API. The attack surface for credential exposure is minimal and contained.

CLI in API Mode

The CLI always operates as an HTTP client to the API server. One behavior regardless of context — local development, production, a harness calling it in subprocess, an engineer running commands remotely. The API server must be running for any CLI operation. This ensures one authentication path, one permission enforcement path, and no behavioral differences between environments.

Dispatch Pattern

When an entity saves and watches fire, the resulting messages are written to the queue in the same transaction. The question: how do durable workflows get started for associate-eligible messages?

Not inside the transaction — starting an external workflow is a network call, and coupling it to the entity save would make saves depend on the workflow engine's availability.

The pattern is optimistic dispatch plus sweep backstop. After the transaction commits, the API server fires and forgets a workflow start for each associate-eligible message. In the normal case, the associate starts processing within seconds. If the dispatch fails — workflow engine blip, API server crash between commit and dispatch — the Queue Processor's sweep catches it within seconds and dispatches. Immediate in the happy path. Automatic recovery in the failure path.

Deployment

The OS deploys to a managed hosting platform in a single region, co-located with the database and workflow engine for single-digit millisecond latency between services. Internal services communicate over an encrypted private network. Public-facing services get automatic TLS certificates.

Rolling deployments ensure zero downtime. New instances start, pass health checks, and begin accepting traffic before old instances drain. Entity type changes — new entity definitions, modified definitions — trigger a rolling restart so the kernel reloads definitions from the database.

For local development: the API server runs with hot reload, the workflow engine runs locally, the UI runs its own dev server, and the CLI connects to the local API. The database is a remote development cluster — no local database needed.

Production Requirements

WebSocket keepalive. The hosting proxy drops idle WebSocket connections after 60 seconds. All WebSocket handlers — the real-time UI channel, the chat harness — must send ping frames every 30-45 seconds. Without this, connections die silently.

External health monitoring. The hosting platform monitors during deployment but not after. External uptime checks are required from day one — HTTP checks for the API server, heartbeat records for internal services.

Connection pool sizing. Each service explicitly configures its database connection pool to stay within the database tier's connection limits. Defaults are tuned per service based on query volume.

Log shipping. Structured logs ship to an external observability platform. The hosting platform's built-in logs serve as a quick debug console with limited retention.

Web application firewall. Before the first external customer, a WAF sits in front of the production API for bot protection, DDoS absorption, and rate limiting.

Cost

Infrastructure cost at MVP is approximately $200/month — database, workflow engine, compute, file storage, and monitoring. This scales to approximately $1,700/month at 50 customers. The architecture is cost-proportional — compute and database tier scale with usage, not with provisioned capacity.

LLM cost for associate processing is separate and per-customer. The `--auto` pattern keeps it proportional to edge-case complexity, not total volume.

---

8. Development and Operations

Repository Structure

The OS is one repository. A modular monolith — clear internal boundaries, one deployable unit.

The kernel code lives in one directory: the entity framework, message handling, watch evaluation, rule engine, capabilities library, authentication middleware, API auto-registration, CLI auto-registration, queue processor, and workflow definitions.

Seed files — standard entity templates, default skills, reference roles — live in their own directory. Harnesses — the glue code per runtime kind and framework — live separately, each producing its own

deployable image. The base UI is its own directory with its own build. Tests are organized by layer: unit, integration, end-to-end.

A CLAUDE.md file at the repository root defines conventions for any session working on the codebase — how to add an entity, how to add a capability, how to add a test, how to run locally, naming conventions, the trust boundary, the API-mode CLI rule.

Testing Strategy

Three layers, each with a different scope and cost:

Unit tests cover individual functions in isolation — entity class creation from definitions, rule evaluation, condition matching, state machine transitions, authentication method validation, token signing. Fast, no external dependencies. Run on every commit.

Integration tests cover multi-component flows — entity save triggers watch evaluation and produces a message in the queue, rule evaluation with lookups, authentication flow end-to-end, bulk operations across batches. These use a real development database and a local workflow engine. Run on every pull request.

End-to-end tests cover full scenarios drawn from the design validation — a complete email processing pipeline, a consumer-facing autonomous transaction, a scheduled bulk update with downstream actor processing. These use the development database and workflow engine. Run before deployment and on demand.

Entity behavior is the most-tested layer. Every entity type gets auto-generated test scaffolding covering CRUD operations, state machine transitions, and permission enforcement.

CI/CD

Push to the main branch triggers the pipeline: lint, unit tests, integration tests, build the image, deploy via rolling update. End-to-end tests run on a schedule or on demand, not on every push.

Parallel AI Sessions

The OS is built by one architect with AI — multiple parallel sessions, each in its own isolated workspace. Independent work — different entity types, different capabilities, different tests — runs in parallel and merges independently. Shared infrastructure — the base entity framework, the API server, the auth middleware — is single-session work. The CLAUDE.md defines patterns all sessions follow. Any session that touches shared infrastructure updates the conventions if they change.

Customer Onboarding

Three tiers of onboarding, with Tier 2 as the primary path for the current model:

Tier 1 (out of the box). Customer self-service. Clone from a template organization, configure through the UI or CLI, go live.

Tier 2 (configured). A forward-deployed engineer works inside the customer's organization through a platform admin session. The checklist: create the organization, define entities (or clone from templates), configure per-org rules and lookups and capability activations, set up integrations, create roles with watches, create associates with skills, create human actors and assign roles, test the pipeline end-to-end in a staging organization, deploy to production, monitor and tune rules based on real data.

Tier 3 (developer). Self-service signup, API key, build with the CLI and API, deploy on the platform.

Monitoring

The base UI provides the primary monitoring surface — queue depths, entity state distributions, integration health, runtime status, active attentions. The CLI provides the same data for scripted checks and automation. The external observability platform provides trace-level debugging, latency analysis, and error rate tracking. The hosting platform and database dashboards provide infrastructure-level visibility.

Debugging Workflow

When something goes wrong:

1. **Identify** — ops sees it in the base UI (queue growing, integration error, bottleneck forming) or an external monitoring alert fires.
2. **Locate** — trace the entity's recent changes and cascade membership through the CLI or UI.
3. **Diagnose** — drill into specific execution spans in the observability platform — timing, errors, rule evaluation results in the changes collection.
4. **Act** — fix the root cause through the CLI or UI: update a rule, rotate a credential, restart a service, reassign work to a human.
5. **Verify** — watch the fix propagate through the live base UI.

Platform Upgrades

Kernel capability upgrades declare configuration schema versions. Entity definitions store which version they use. Upgrades include migration scripts. A dry-run previews what would change. The upgrade applies migrations, auditable and rollbackable. Zero downtime through rolling deployment of the new kernel code.

Backup and Disaster Recovery

The database provides automatic backups and point-in-time recovery. The workflow engine's state is fully durable. File storage has versioning enabled. Configuration is its own backup — the changes collection is the complete history, and organization export produces YAML snapshots that can be imported to restore. No additional backup infrastructure is needed at MVP.

---

9. Transition and Coexistence

Separate Systems

The current Indemn platform and the new OS are completely separate. Different codebases, different infrastructure, different databases. They don't share resources and they don't need to communicate.

	Current Platform	New OS
Codebase	Multiple repositories	One repository
Database	Existing database cluster	New database on separate cluster
Compute	Existing deployment	New deployment project
Maintained by	Current engineering team	Craig, with AI
Serves	All current customers	New customers, then migrated existing ones

The Coexistence Model

They run side by side indefinitely. No shared state, no message passing, no bridge layer. The current platform keeps serving current customers as-is. The OS is built and deployed independently.

Nobody is asked to stop what they're doing. The OS is additive work until it's ready. The team gradually transitions as the OS absorbs workloads.

When Customers Move

New customers go on the OS from the start, once it's production-ready. Existing customers migrate when the OS can serve them AND the migration has a concrete benefit. Not forced, not on a timeline.

Each migration is its own project: model the customer's workflows on the OS, set up integrations, run both systems in parallel with the same inputs and compare outputs, cut over when confident, decommission the old configuration.

What the Current Platform Teaches the OS

Every bespoke customer implementation is R&D for the OS. The patterns discovered in custom builds — entity lifecycles, watch-driven pipelines, integration adapters, the `--auto` pattern — are baked into the kernel architecture. The current platform is the proving ground. The OS is the generalization.

---

10. Dependencies and Resilience

The Dependency Map

The OS depends on external services. Each has a different failure mode and a different impact.

Dependency	What Depends On It	If It Goes Down
Database	Everything — entities, queue, auth, configuration	Full outage. Protected by the database provider's high-availability architecture — multi-node replica set with automatic failover.
Workflow Engine	Associate execution, bulk operations, scheduled tasks	Associates stop. Humans continue through the unified queue. Backlog replays automatically on recovery.
LLM Providers	Associate reasoning (the AI fallback in the `--auto` pattern)	LLM-dependent processing stops. Deterministic operations continue — rules, lookups, state transitions all work without AI. Different associates may use different providers, so a single provider outage only affects associates on that provider.
Secrets Manager	Credential resolution for integrations, authentication	New credential fetches fail. Existing sessions continue — access tokens are self-contained. Cached credentials serve until TTL expires.

File Storage	PDFs, attachments, certificates, generated documents	File operations fail. Everything else works.
Compute Platform	All services	Full outage. Automatic recovery — workers are stateless, the database has the data, the workflow engine has execution state.
Observability Platform	Monitoring, traces	Monitoring goes blind. Zero application impact. Trace export is fire-and-forget.

The Key Insight

The database is the only single point of failure. Everything else degrades gracefully:

- Workflow engine down → humans continue through the unified queue. This is the design advantage of the unified queue — work doesn't disappear when the automation layer goes down.
- LLM provider down → the `--auto` pattern continues deterministic processing. Rules still evaluate. Lookups still resolve. Only the reasoning fallback is unavailable.
- Secrets manager down → cached credentials serve. Sessions validated by self-contained tokens continue without interruption.
- File storage down → only file operations affected. Entity processing, messaging, and queue operations are unaffected.
- Compute platform down → stateless workers recover on restart. The database and workflow engine preserve all state.

Multi-Provider LLM Resilience

The OS is LLM-provider-agnostic. Associates specify their model in configuration. Multiple providers can coexist across different associates in the same organization. A single provider outage affects only the associates configured to use that specific provider. This is a structural resilience advantage — not a feature that needs to be built, but a natural consequence of making the model configurable per associate.

What Needs to Be Built

- **Credential caching with TTL** in the API server — fetch from the secrets manager at startup, cache in memory, refresh periodically, serve from cache if refresh fails.

- **Retry policies** on all workflow activities — exponential backoff, maximum attempts, timeout per activity.
- **Health endpoint** that checks connectivity to all dependencies and returns degraded status for non-critical failures.
- **Graceful degradation in the base UI** — status banner for degraded dependencies, not a full error page.

---

11. Build Sequence

Approach

Build by hand first. Extract the framework from what works. Then parallelize.

The OS is not built by designing a framework in the abstract and then instantiating entities from it. It's built by hand-crafting the first few entities end-to-end — experiencing every decision, every edge case, every interaction between primitives — and then extracting the reusable framework from the working code. The framework emerges from usage, not from theory.

Each phase produces something that works and can be validated. Nothing is built speculatively. Each phase unblocks the next.

Phase 0: Development Foundation

What it produces: The repository, the conventions, and the environment for building everything else.

- Repository structure following the modular monolith design
- CLAUDE.md for the codebase — conventions for every session that touches the code
- Development environment — local API server, local workflow engine, connection to development database
- CI pipeline — lint, test, build, deploy
- Deployment configuration for the kernel image and supporting services

Acceptance criteria: A developer (or AI session) can clone the repo, start the local environment, run the test suite, and deploy to the hosting platform. The conventions are documented. The pipeline works.

Unblocks: Everything.

Phase 1: Kernel Framework

What it produces: The base entity class and the auto-generation machinery — the kernel that everything else is built on.

This phase hand-builds the entity framework by implementing the first entity type end-to-end:

- Entity definitions as data — stored in the database, loaded at startup, generating classes dynamically
- State machine enforcement on entities that define one
- The auto-generated CLI — CRUD operations, state transitions, exposed operations, all derived from the entity definition
- The auto-generated API — same operations, same permissions, mirroring the CLI
- The auto-generated skill — markdown documentation produced from the entity definition
- Organization scoping — every query scoped, no way to bypass
- The changes collection — every mutation recorded with field-level detail
- Authentication middleware — token validation, permission enforcement, session management
- The message queue — entity saves produce messages atomically, watches evaluate, messages route to roles
- The condition evaluator — shared by watches and rules, JSON format
- Kernel capabilities — at least one (for example, auto-classify with the `--auto` pattern) proven end-to-end
- Rules and lookups — created via CLI, evaluated by capabilities, stored as data

Acceptance criteria: One entity type works end-to-end. Define it through the CLI. Its API exists. Its CLI commands exist. Its skill is generated. Create an instance. Transition its state. Watch evaluation fires. A message appears in the queue. A rule evaluates deterministically. The changes collection records every mutation. Organization scoping prevents cross-tenant access. Authentication works for human and service token sessions.

This is the hardest phase. Everything after it builds on what this phase proves.

Unblocks: All domain entity work, associate execution, the base UI.

Phase 2: Associate Execution

What it produces: The ability for AI actors to process work from the queue.

- Integration with the durable workflow engine — the generic claim-process-complete workflow

- Skill loading — associates read their skills from the database at invocation
- The `--auto` pattern working end-to-end — capability tries rules, returns `needs_reasoning`, associate's skill provides the AI fallback
- Scheduled task execution — cron triggers create queue items, same path as watch-triggered work
- Direct invocation for real-time — queue entry created for visibility, associate invoked directly for latency

Acceptance criteria: An associate configured with a role and skill claims a message from the queue, processes it by executing CLI commands through the `--auto` pattern, changes entity state, and the resulting messages propagate through watches. A scheduled associate runs on its cron and processes work through the same path. Direct invocation works for latency-sensitive triggers.

Unblocks: Any use case that involves AI processing.

Phase 3: Integration Framework

What it produces: The ability to connect the OS to external systems.

- The Integration kernel entity — provider, credentials, ownership, lifecycle
- Adapter dispatch — resolving the right adapter from the integration's provider
- Credential resolution — actor-level first, org-level fallback, secrets manager for storage
- At least two working adapters proving both outbound and inbound patterns (for example, an email provider for outbound fetch/send, and a payment provider with inbound webhooks)
- The webhook endpoint — generic handler that dispatches to the integration's adapter for validation and parsing

Acceptance criteria: An integration is created via CLI with credentials stored securely. An entity operation resolves the integration, fetches credentials, and successfully communicates with the external system. An inbound webhook is received, validated, and parsed into entity operations. Credential rotation works without downtime.

Unblocks: Any use case that involves external systems — which is nearly all of them.

Phase 4: Base UI

What it produces: The auto-generated operational surface.

- Entity list views and detail views derived from entity definitions
- Queue view for each actor's pending work

- State machine visualization — current state plus transition buttons
- Forms auto-generated from entity fields
- Kernel entity views — actors, roles, integrations, runtimes, attentions, sessions
- Real-time updates via change streams filtered by current view
- The assistant — default associate per user, persistent input, context-aware

Acceptance criteria: An operator can log in, see their queue, drill into entities, take actions (transitions, exposed operations), see the system update in real time, and interact with the assistant to perform operations through natural language. Adding a new entity type to the system makes it appear in the UI without any UI code changes.

Unblocks: Operational use of the OS by humans. Customer onboarding through the UI.

Phase 5: Real-Time

What it produces: The ability to run live conversations — voice, chat, SMS — through the OS.

- The Attention kernel entity — active working context, heartbeat, TTL
- The Runtime kernel entity — execution environment, instance tracking, capacity
- At least one harness proving the pattern (for example, a chat harness bridging a WebSocket server to the kernel via CLI)
- Scoped watches — active context resolution for mid-conversation event delivery
- Handoff — transfer between actors on a live interaction

Acceptance criteria: A consumer opens a chat session. An associate handles the conversation in real time. Entity operations during the conversation update state and trigger watches. A scoped event (for example, a payment completing) reaches the running associate mid-conversation. A human can observe and take over the conversation. The harness uses the CLI for all OS interactions.

Unblocks: Consumer-facing products. EventGuard-style autonomous flows.

Phase 6: Dog-Fooding

What it produces: Indemn running its own operations on the OS.

This phase models Indemn's internal operations as a domain on the kernel — the first real-world validation that the OS works for a complete business. The specific scope is determined when this phase begins, but the intent is: Indemn uses the OS for its own work, proving the platform before putting customers on it.

Acceptance criteria: The Indemn team uses the OS daily for actual work. Associates process real data. The base UI is the operational surface. Issues found are fixed in the kernel. The system is stable enough that the team relies on it.

Unblocks: Confidence to put external customers on the platform.

Phase 7: First External Customer

What it produces: A real insurance business running on the OS.

The first external customer is onboarded through the domain modeling process — entities defined, roles configured, rules created, associates deployed, integrations connected. This is the full test of the Tier 2 onboarding path.

Acceptance criteria: A real insurance workflow runs entirely on the OS, processing real data, with real users and real associates working the same queues. The customer's operations are modeled, not custom-coded. The system handles the customer's volume and complexity. Issues found are addressed in the kernel or the domain configuration.

Unblocks: Scaling to additional customers. The proof that the OS works.

What Comes After

Each subsequent customer is faster than the last. Standard entity templates emerge from repeated use. Skills get reused and refined. Rule patterns become starting points. The adapter library covers more providers. The intelligence flywheel begins — data from each customer improves the models that serve all customers.

The build sequence is complete when the OS reaches the point where new customers are configuration, not construction. That's the operating system.

This document is the source of truth for the design of the Indemn Operating System. Implementation specifications for each phase are produced separately, with research and detailed design, when building begins.